

PROGRAMMING ASSIGNMENT 1

TAs : Alpay TEKİN

Due Date : 12.04.2026 (23:00:00)

Programming Language: Java 8 (Oracle)

Dungeon Simulation



1 Introduction

In this assignment, you will implement a dungeon-escape simulation in Java. A player character starts in a dungeon represented as a two-dimensional grid of rooms and must navigate from the starting position to the exit room. Before reaching the exit, the player must pick up the key stored in a designated Key Room.

2 Problem Description

The dungeon is represented as a rectangular grid. Each cell of the grid contains a room. The player starts at a given coordinate with full health and tries to reach the target coordinate.

The program must find the best valid path according to the following rules:

- The path with the **maximum remaining health** must be selected.
- If multiple paths end with the same remaining health, the **shorter path** must be selected.

- The cost of moving from one room to another is 1.
- If there is still a tie, the deterministic traversal order will determine the selected path.

2.1 Dungeon Description

The dungeon is a grid of N rows and M columns. Each cell of the grid is a room. The player begins at a fixed starting cell (given in the input file) with a given initial health value. The player moves through adjacent rooms one step at a time. You must consider followings:

- Movement is restricted to the four cardinal directions in order: Up, Right, Down, and Left.
- The player cannot move outside the grid boundaries.
- A room may be visited at most once per path (no cycles).
- The game ends successfully when the player stands in the Exit Room after having collected the key.

2.2 Room Types

Every room in the dungeon belongs to exactly one of the following types:

Table 1: Room Types and Their Effects

Room Type	Token	Effect on Player
HealingRoom	H	Player gains +15 HP upon entry.
TrapRoom	T	Player receives −20 HP damage upon entry.
MonsterRoom	M	Player receives −30 HP damage upon entry.
ArtifactRoom	A,L,B	Contains one artifact; artifact effect is applied immediately upon entry (see Table 2).
KeyRoom	K	Player collects the key. No HP change.
EmptyRoom	E	No effect on the player.
ExitRoom	X	Goal room. Player must hold the key to complete the dungeon.

2.3 Artifacts

An ArtifactRoom contains exactly one artifact. Artifacts are collected immediately upon entering the room and take effect according to their type:

Notes on shield artifacts (ArmorRelic, BoostRelic):

- Only one shield is active at a time. If the player picks up a second shield before the first is consumed, the new shield replaces the old one.

Table 2: Artifact Types and Their Effects

Artifact	Token	Effect
RelicHeal	L	Immediately restores +10 HP upon collection.
ArmorRelic	A	Activates a one-time shield that reduces the next incoming damage by 6 HP . If a shield is already active, it is replaced by this one.
BoostRelic	B	Activates a one-time shield that reduces the next incoming damage by 4 HP . If a shield is already active, it is replaced by this one.

Note: Shield artifacts (ArmorRelic, BoostRelic) are consumed upon the next damage event. Effective damage = $\max(0, \text{base_damage} - \text{shield_value})$.

- A shield is consumed upon the next damage event (TrapRoom or MonsterRoom). The damage applied is: $\text{effectiveDamage} = \text{baseDamage} - \text{shieldValue}(\text{minimum } 0)$.

2.4 Backtracking

Backtracking is a recursive problem-solving technique that explores all possible paths through a search space. The idea is straightforward: the algorithm moves forward step by step, and whenever it reaches a dead end — either because the player has run out of health, or because there are no more unvisited neighbours to explore — it *backtracks* to the previous step and tries a different direction.

In this assignment, the algorithm starts from the player's initial room and explores the dungeon room by room. At each room, it tries to move to a neighbouring room in the following fixed order: **Up, Right, Down, Left**. This ordering is enforced to make the search deterministic, meaning that two correct implementations will always explore paths in the same sequence and produce identical output.

A path is considered *valid* if the player collects the key before reaching the exit and never runs out of health along the way. Among all valid paths, the algorithm keeps track of the *best* solution found so far — first preferring the path that leaves the player with the most health, and in case of a tie, preferring the shorter path.

Every time the algorithm steps into a room, it applies that room's effect to the player (gaining health, taking damage, picking up the key, etc.). When it backtracks out of that room, it *undoes* all those changes so that the player's state is exactly as it was before entering the room. This *state restoration* step is critical: without it, effects from one explored branch would incorrectly carry over into another.

The general structure of a backtracking algorithm is shown in Algorithm 1.

Algorithm 1 Generic backtracking

```
1: procedure BACKTRACK(current state)
2:   if current state is a solution then
3:     Record or update the best solution
4:   return
5:   for each available choice from the current state do
6:     if the choice is valid then
7:       Apply the choice                                ▷ modify the state
8:       BACKTRACK(new state)
9:       Undo the choice                                ▷ restore the state
```

3 Test and Execution

Your code must be compiled and executed under Java 8 and dev.cs.hacettepe.edu.tr. If your code does not compile and execute under developer server, then you will be graded as 0 for code part even if it works on your own machine. Your program must take input files, and name of the output file. Sample run command is as follows:

```
javac Main.java
java Main input.txt output.txt
```

4 Sample Input and Output

The following illustrates the sample input text file to initialize the dungeon simulation.

Listing 1: Sample input

```
5 5
100
0 0
0 4
E K E M X
E M M E H
E E E E E
T T T T E
E E E E E
```

Here,

- The first row indicates the size of the dungeon. The program will create a 5×5 dungeon according to this input.
- The second row indicates the starting health of the player, which is 100 for the given input file.
- The third row indicates the start position of the player, and the fourth row indicates the location of the exit room.
- The remaining rows describe the type of each room (tile) in the dungeon. Refer to Table 1 and Table 2 for the letter that represents each room type.

The following illustrates a sample output file that your program is expected to generate.

Listing 2: Sample output

```
RESULT: SUCCESS
MAX_HEALTH: 100
REMAINING_HEALTH: 85
STEPS: 6
PATH:
(0,0,100) -> (0,1,100) -> (0,2,100) -> (0,3,70) -> (1,3,70) ->
(1,4,85) -> (0,4,85)
```

Here,

- The first row indicates the status of the search: either **SUCCESS** or **FAILURE**.
- The second row indicates the initial health of the player.
- The third row indicates the remaining health upon reaching the exit room.
- The fourth row indicates the number of steps (room transitions) taken to reach the exit room.
- The last row describes the final path to the exit room as a sequence of tuples. Each tuple is of the form (r, c, h) , where r and c are the row and column indices of the room, and h is the player's remaining health after entering that room and applying its effect. For example, $(0, 3, 70)$ means the player is at row 0, column 3, with 70 HP remaining.

The following examples illustrate the expected output for failure scenarios: cases where the dungeon contains no key, or where the player cannot reach the exit without dying

Listing 3: Sample output when there is no key.

```
RESULT: FAILURE
MAX_HEALTH: 100
REASON: No key found.
```

Listing 4: Sample output when the player cannot reach the exit without dying.

```
RESULT: FAILURE
MAX_HEALTH: 30
REASON: Player died before reaching the exit.
```

5 Grading Policy

Task	Grade
Class Implementations using OOP Principles	60*
Correct Output	30**
Comments in JavaDoc Style	10***
Total	100

***In order to receive full credit for the class implementation, you must apply OOP principles — namely encapsulation and inheritance — correctly. Failure to do so will result in either no credit or partial credit for that section.**

****Even though producing correct output and commenting seems like enough to get full credit, you must obey to the given rules in the PDF (for example using concept of OOP and four pillars of OOP etc.) otherwise you may face with some point deductions which may result with a grade that is as low as zero. There will be two overall multipliers about quality of your OOP and clean code separately which will vary in between 0 and 1! Note that there may be any other multipliers or point deductions in case of violation of the rules.**

***** The score of the clean code comment part will be multiplied by your overall score (excluding clean code comment part) and divided by the maximum score that can be taken from these parts. Say that you got 45 from all parts excluding clean code comment part and 10 from clean code comment part, your score for clean code comment part is going to be $10 \cdot (45/90)$ which is 5 and your overall score will be $45 + 5 = 50$.**

6 Submit Format

File hierarchy must be zipped before submitted (Not .rar, only .zip files are supported by the system).

```
-b<studentID>.zip  
-Main.java and other java files
```

Late Policy

You have two days for late submission. You will lose 10 points from maximum evaluation score for each day (your submitted study will be evaluated over 90 and 80 for each late submission day). You must submit your solution in at the most two days later than submission date, otherwise it will not be evaluated. Please do not e-mail to me even if you miss the deadline for a few seconds due to your own fault as it would be unfair for your friends, e-mail submissions will not be considered if you do not have a valid issue.

Notes and Restrictions

- Your code must be able to execute on our department's developer server (dev.cs.hacettepe.edu.tr).
- You must use JavaDoc commenting style for this project, and you must give brief information about the challenging parts of your code, do not over comment as it is against clean code approach. Design your comments so that if someone wants to read your code they should be able to easily understand what is going on. You can check here to access Oracle's own guide about JavaDoc Sytle.
- **Use inheritance, and encapsulation where it makes logical and practical sense. Otherwise, you risk losing points for not properly applying OOP principles.**

- You must obey given submit hierarchy and get score (1 point) from the submit system.
- Do not miss the submission deadline.
- You can benefit from Internet sources for inspiration but do not use any code that does not belong to you.
- You can discuss high-level (design) problems with your friends but do not share any code or implementation with anybody.
- Source code readability is a great of importance. Thus, write READABLE SOURCE CODE, comments, and clear MAIN function. This expectation will be graded as “clean code”.
- Use UNDERSTANDABLE names for your variables, classes, and functions regardless of the length. The names of classes, attributes and methods must obey to the Java naming convention. This expectation will be graded as “coding standards”.
- You can ask your questions through course’s Piazza group, and you are supposed to be aware of everything discussed in the Piazza group. General discussion of the problem is allowed, but **DO NOT SHARE** answers, algorithms, source codes and reports.
- All assignments must be original, individual work. Duplicate or very similar assignments are both going to be considered as cheating.